

TraceNet — Team Onboarding & Claude Prompts

This doc has four parts:

1. **What TraceNet actually is + a plain-English glossary** — no prior ML knowledge needed
2. **The final plan / roadmap** — what's left, in what order, over what timeframe
3. **Setup steps** every teammate follows once (clone + run)
4. **A ready-to-paste Claude prompt for each role** — copy the whole block for your role into a new Claude chat. It has everything Claude needs to help you immediately: what TraceNet is, what your file currently does, exactly what to change, why, and how it affects the team.

PART -1 — What is TraceNet? (Plain English)

Imagine a criminal wants to hide stolen or illegal money so it looks clean. One way to do this is called **money muling**: split the money into small chunks, move it through several different people's bank accounts, across different channels (mobile app, ATM, wire transfer, UPI), so no single bank ever sees the full picture. Each bank only watches its own little piece — that's the blind spot criminals exploit.

TraceNet's idea: instead of watching each transaction in isolation, draw the *whole picture* as a map — every person is a dot, every transaction is a line connecting two dots. Once you can see the whole map, patterns jump out that were invisible before — like five separate accounts all quietly funneling money to the same one account within minutes of each other.

TraceNet then uses an AI model to look at this map and flag which parts of it look like money laundering, blocks the suspicious transaction, and writes up a short report automatically explaining exactly why it made that decision — the kind of report a bank compliance officer would otherwise have to write by hand.

In one sentence, for anyone who asks: *"TraceNet is an AI system that maps out how money moves between accounts across different banking channels, and uses that map to catch laundering patterns that traditional systems — which only look at one channel at a time — would miss."*

PART -0.5 — Glossary: every technical term explained simply

Use this as a reference whenever a term feels unfamiliar — in a chat with teammates, your supervisor, or your own Claude prompt. No prior ML background assumed.

The core AI concept

Graph — not a bar chart. In this context, a graph means a network of dots connected by lines. Each dot is called a **node**, each line is called an **edge**. In TraceNet: nodes = people/accounts, edges = transactions between them.

GNN (Graph Neural Network) — a type of AI model built specifically to learn from graphs, instead of plain tables of numbers. A normal AI model looking at a bank account only sees that account's own numbers (balance, transaction count). A GNN also looks at *who that account is connected to*, and who those connections are connected to — like judging someone partly by the company they keep, not just by themselves.

GraphSAGE — the specific type of GNN TraceNet uses. Think of it as a method where each node repeatedly "asks its neighbors" for information and blends it with its own, a few rounds in a row, so it ends up with a rich picture of its surroundings, not just itself. It's simply one well-known "flavor" of GNN, chosen because it works well and scales to large graphs.

Node embedding — after the GNN processes a node, it produces a list of numbers that summarizes "everything relevant about this node and its neighborhood" in a compact form. Think of it as a fingerprint the model builds for each account.

Layer (in a neural network) — one step of processing. TraceNet's GNN has 3 layers, meaning information passes through 3 rounds of "gather info from neighbors and refine" before the model makes its final decision.

Training — the process of showing the model lots of labeled examples ("this account was a mule," "this one was clean") so it learns the patterns on its own, rather than being told exact rules.

Features — the raw input numbers describing something. For a transaction: amount, time, channel, etc. "Feature engineering" just means deciding which numbers to feed the model and how to calculate them.

The comparison / evaluation side

XGBoost — a different, older, very reliable type of AI model that works great on plain tables of numbers, but — importantly — **cannot see graph relationships**. It's used here as the "simpler alternative" to compare against, to prove the GNN's graph-awareness actually earns its keep.

Baseline — a simpler method used as a point of comparison, to prove your fancier method is actually worth it. XGBoost is TraceNet's baseline.

Accuracy — % of all predictions the model got right. Can be misleading if one class is rare (see "class imbalance" below).

Precision — of everything the model flagged as suspicious, what % actually was suspicious. Low precision = lots of false alarms.

Recall — of everything that actually was suspicious, what % did the model catch. Low recall = the model misses real criminals.

F1 Score — a single number that balances precision and recall together, useful when you can't just look at one of them alone.

Confusion matrix — a small table showing exactly how many predictions were: correctly flagged, correctly cleared, wrongly flagged (false alarm), and wrongly cleared (missed criminal). More informative than a single accuracy number.

Class imbalance — when one outcome is much rarer than the other. TraceNet's synthetic data has only ~92 "mule" examples out of ~5000 total — meaning a lazy model could get 98%+ accuracy just by *always* guessing "not a mule." This is exactly why accuracy alone isn't convincing, and why precision/recall/F1 matter more.

Synthetic data — fake data generated by a script (TraceNet currently uses a library called "Faker") instead of real-world records. Easy to control, but doesn't prove the model works on messy real data.

Elliptic dataset — a real, public dataset of actual Bitcoin transactions, labeled as legal or illicit by researchers. We're using it to test TraceNet against real-world data instead of only data we made up ourselves.

The software/engineering side

Backend — the part of the app that does the actual thinking: runs the AI model, makes decisions, stores logic. In TraceNet, this is `backend/api.py`, built with a tool called **FastAPI**.

Frontend — the part of the app the user actually sees and clicks on. In TraceNet, this is `frontend/app.py`, built with a tool called **Streamlit** (it turns Python code into a web dashboard without needing to write HTML/CSS/JavaScript by hand).

API / API endpoint — a specific "address" the frontend calls to ask the backend to do something. E.g. `/score_transaction` is the address the frontend calls whenever it needs the backend to check whether a transaction is suspicious. Think of it like a restaurant menu — each endpoint is one specific thing you can order.

LLM (Large Language Model) — the general category of AI that writes/understands human language (like ChatGPT or Claude). TraceNet uses one locally (see below) to auto-write the suspicious activity reports in plain English.

Ollama — a free tool that lets you run an LLM directly on your own computer instead of calling an online service. TraceNet uses it to run a small model called **Qwen 2.5** locally, so report-writing doesn't depend on an internet connection or paid API.

SAR (Suspicious Activity Report) — a real, standard document type banks are legally required to file when they detect potential money laundering. TraceNet auto-generates a SAR-style report whenever it blocks a transaction.

Hashing / anonymization (SHA-256) — a way to scramble sensitive data (like a person's real account ID) into a scrambled code that can't be reversed back to the original, while still being consistent (same input always makes the same scrambled output). This lets TraceNet share "this account looks suspicious" between banks without exposing anyone's real identity.

Exception handling / bare except: — code that deals with errors. A "bare except" means "catch literally any error and ignore it silently" — which is risky because it can hide real bugs, not just expected problems. Good exception handling catches *specific*, expected error types and handles each one properly.

requirements.txt — a plain text file listing every external tool/package the project needs installed to run. If it's out of date, other people's setups break even though the code itself is fine.

Git branch — your own private copy of the code to work on, so you don't accidentally break what teammates are doing. You "merge" it back into the shared `main` version once your part works.

PART 0 — Final Plan & Roadmap

The situation in one line

The core platform (data pipeline, GNN model, backend API, frontend dashboard) already works end-to-end. What's missing is the academic rigor a graded project needs: proof the model works on real data (not just synthetic), proof it beats simpler approaches, automated tests, and the written report.

The new addition: validating on a real dataset

So far, TraceNet has only been trained and tested on **synthetic data** — fake users and transactions generated by a script. That's a real weakness, because it's easy to hit 99%+ accuracy on data you made up yourself. To fix this, we're adding one real, public, well-known dataset to validate against: the **Elliptic dataset** — a real Bitcoin transaction graph where each transaction is labeled licit or illicit, published specifically for AML/fraud research.

Where to get it / who owns it:

- Source: <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set> (free, requires a Kaggle account to download)
- Format: 3 CSV files (~203,769 transaction nodes, ~234,355 edges, 166 features per node, ~2% labeled illicit, ~21% labeled licit, rest unlabeled, spanning 49 time steps)
- **Owned by: the Evaluation & Baseline teammate** (Role 2 below) — this pairs naturally with the baseline comparison work, since it's the same job: "run our existing pipeline against a second data source and report what happens"

8-week roadmap (~2-3 hrs/week per person)

Week	What happens
1	Group call (~30 min): everyone gets the plain-English explanation of the Elliptic dataset and why we're adding it (so everyone can explain it later, not just one person). Roles assigned, everyone clones the repo and confirms it runs locally.
2	Evaluation teammate downloads the Elliptic dataset and explores its structure vs. our synthetic data. Backend teammate starts the quick fixes (requirements.txt, exception handling). Frontend teammate starts UI polish.
3-4	Evaluation teammate adapts the data-loading code to accept the Elliptic dataset, retrains/evaluates both the GNN and the XGBoost baseline on it. Backend teammate writes automated tests. Frontend teammate builds the interbank-sharing view. Literature review teammate gathers sources in parallel.
5	Evaluation teammate finishes the comparison: synthetic-data results vs. real-data results, GNN vs. XGBoost, with confusion matrices. This becomes the "Results & Evaluation" report section.
6	Documentation teammate wires results into a polished live demo (<code>scripts/simulator.py</code>) and starts compiling the report. Everyone reviews/tests each other's branches.
7-8 (buffer)	Report writing and compilation, final testing pass, presentation slides, dry-run of the demo.

Task ownership at a glance

Task	Owner	Status
GNN model + training (synthetic data)	You (Model Lead)	Done
Backend core logic	You (Model Lead)	Mostly done
requirements.txt fix + exception handling	Backend/Testing teammate	Not started
Automated tests	Backend/Testing teammate	Not started
Real dataset (Elliptic) sourcing + integration	Evaluation & Baseline teammate	Not started
Baseline comparison (XGBoost vs GNN, synthetic + real data)	Evaluation & Baseline teammate	Not started
Frontend polish + interbank sharing view	Frontend teammate	Partially done
Literature review	Literature Review teammate	Not started
Live demo polish + report compilation + ethics section	Documentation teammate	Not started

PART 1 — Setup (everyone does this first)

Written for Windows, assuming nothing is installed except VS Code. Every command below is meant to be run **one at a time** — press Enter and wait for it to finish before running the next one. Don't paste two commands on one line.

Step 0: Check Python is installed

```
py --version
```

You should see something like `Python 3.10.x` or `Python 3.11.x`. If instead you see `'py' is not recognized`, Python isn't installed:

5. Go to <https://www.python.org/downloads/>
6. Download and run the installer
7. **Important:** on the first install screen, tick the box **"Add python.exe to PATH"** at the bottom before clicking Install
8. Close and reopen VS Code completely, then re-run `py --version` to confirm

Step 1: Check Git is installed

```
git --version
```

If you see `'git' is not recognized`, install it from <https://git-scm.com/download/win> (default options are fine during install), then close and reopen VS Code.

Step 2: Clone the repo

```
git clone https://github.com/Aviralsahu960/TraceNet.git
```

```
cd TraceNet
```

Step 3: Create a branch for your own work

```
git checkout -b your-name-your-task
```

Replace `your-name-your-task` with something real, e.g. `git checkout -b priya-baseline-eval`. Don't push to `main` directly.

Step 4: Create a virtual environment

We're using Python's built-in `venv` instead of `conda` — it comes with Python already, so there's nothing extra to install.

```
py -m venv tracenet_env
```

Step 5: Activate the virtual environment

```
tracenet_env\Scripts\activate
```

Your terminal line should now start with `(tracenet_env)`. If you get an error saying **"running scripts is disabled on this system"**, see the Troubleshooting section below — this is a common Windows security setting, not a mistake you made.

Step 6: Fix requirements.txt (do this before installing anything)

Open `requirements.txt` in VS Code and make it match this exactly:

```
faker
pandas
networkx
```

```
scikit-learn
xgboost
fastapi
uvicorn
streamlit
streamlit-agraph
python-dotenv
ollama
requests
```

(This removes `google-generativeai`, which was listed but never actually used, and adds `ollama` and `requests`, which the code needs but weren't listed.) Save the file.

Step 7: Upgrade pip

```
python -m pip install --upgrade pip
```

Step 8: Install PyTorch (CPU version — works on every laptop)

```
pip install torch torchvision torchaudio
```

Only if you specifically have an NVIDIA GPU and know you have CUDA set up, use this instead of the line above:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

If you're not sure, use the plain CPU version — it always works, just runs a bit slower. This project doesn't need a GPU.

Step 9: Install PyTorch Geometric

```
pip install torch-geometric
```

Step 10: Install everything else from requirements.txt

```
pip install -r requirements.txt
```

Step 11: Confirm the setup works (optional — models are already trained and committed)

```
python scripts/get_real_data.py
```

```
python scripts/train_model.py
```

Step 12: Run the app (needs two terminals open at once)

Terminal 1 — backend:

```
python -m uvicorn backend.api:app --reload
```

Leave this running. **Open a second terminal** in VS Code (Terminal menu → New Terminal), activate the environment again (`tracenet_env\Scripts\activate`), then:

Terminal 2 — frontend:

```
python -m streamlit run frontend/app.py
```

It should automatically open **http://localhost:8501** in your browser. If not, open that link manually.

Step 13: When you're done with your task

```
git add .
```

```
git commit -m "clear description of what you changed"
```

```
git push origin your-name-your-task
```

Then message the group — Aviral will review and merge into `main`.

Troubleshooting — errors you're likely to hit

"running scripts is disabled on this system" when activating the environment

This is a Windows security setting, not a bug. Fix it once per machine: open **PowerShell as Administrator** (right-click Start button → "Terminal (Admin)" or "Windows PowerShell (Admin)"), run:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Type `Y` and press Enter if asked to confirm. Close that window, go back to your normal VS Code terminal, and try Step 5 again.

"'python' is not recognized" but `py` works fine

Windows sometimes only registers the `py` launcher, not `python` directly. Just use `py` instead of `python` in any command that fails this way (e.g. `py -m pip install --upgrade pip`).

A `pip install` command seems to combine two commands into one and fails with "No matching distribution found for install"

This means two separate commands got pasted as a single line by accident. Every command in this guide must be run on its own line, one at a time — copy just one block, press Enter, wait for it to finish, then copy the next.

"Port 8000 (or 8501) is already in use"

Something is already running on that port — usually a previous run of the same app you forgot to close. Close the old terminal window(s) running `uvicorn` or `streamlit`, then try again.

`pip install torch-geometric` seems to hang or takes a long time

This is normal — it's a large package with several components. Give it a few minutes before assuming it's frozen.

Antivirus or Windows Defender blocks/flags something during install

Rare, but if it happens, it's almost always a false positive on a legitimate Python package. Allow it and continue.

PART 2 — Role-specific Claude prompts

Each block below is self-contained. **Copy the entire block for your role**, paste it as your first message in a new Claude chat, then attach/paste the relevant file(s) mentioned when Claude asks. You don't need to explain anything else — the prompt already has full project context.



ROLE 1 — Evaluation & Baseline Comparison

I'm working on a university-graded AI/ML project called TraceNet with a team of 6.

WHAT TRACENET IS:

TraceNet is an Anti-Money Laundering (AML) detection platform. It builds a graph where every bank user is a node and every transaction is an edge. A Graph Neural Network (GraphSAGE, 3 layers: 15→64→32→2) looks at each user's transaction neighborhood to score how "mule-like" (money-laundering-like) their behavior is. If risk is high, the transaction is blocked and an AI (Ollama LLM) writes a short suspicious activity report explaining why.

Tech stack: PyTorch + PyTorch Geometric (GNN), FastAPI (backend), Streamlit (frontend), XGBoost (a second, older model that exists in the repo but isn't currently used for comparison).

MY SPECIFIC TASK (two parts):

PART A — Baseline comparison on our existing (synthetic) data:

I need to build a baseline comparison to prove the GNN is actually better than a simpler approach — this is required for the "Results & Evaluation" section of our report, because right now the project only reports GNN metrics (99.96% accuracy, 100% recall) with no baseline, which examiners will question given the dataset only has 92 positive (mule) examples out of ~5000 nodes — a strong class imbalance.

PART B — Validating on a real, public dataset:

Our data so far is entirely synthetic (generated with the Python "Faker" library), which is a real weakness — it's easy to hit high accuracy on data you made up yourself. I need to also test our pipeline against a real, published dataset used in actual AML research: the Elliptic dataset, a real Bitcoin transaction graph where every transaction is labeled licit or illicit.

- Source: <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set> (free, needs a Kaggle account, comes as 3 CSV files)
- Structure: ~203,769 transaction nodes, ~234,355 directed edges, 166 features per node (94 local features like time/amount/fees + 71 features aggregated from 1-hop neighbors), ~2% labeled illicit, ~21% labeled licit, the rest unlabeled, spanning 49 time steps
- This is graph-shaped data already, so it's a natural fit for our existing GraphSAGE model — the task is adapting our data-loading step to accept it, not building a new model

WHAT ALREADY EXISTS (do not rebuild from scratch):

- models/xgb_model.pkl — an already-trained XGBoost model, currently unused for comparison anywhere in the project
- models/model_config.json — has our synthetic dataset stats (node_count, edge_count, mule_count)
- scripts/train_model.py — shows how the GNN was trained, feature engineering, train/test split logic (I'll paste this)

WHAT I NEED TO CHANGE / BUILD:

1. Load xgb_model.pkl and evaluate it on the same test split used for the GNN on our synthetic data (using only tabular/behavioral features, no graph structure)
2. Load the trained GNN and evaluate it on the same synthetic-data split
3. Produce a comparison table: Accuracy, Precision, Recall, F1, plus a confusion

matrix for both models

4. Download the Elliptic dataset, write a data-loading adapter so our GNN and XGBoost baseline can also be evaluated on it, and produce the same comparison table for the real data
5. Write up an honest comparison of synthetic-data results vs. real-data results – if real-data accuracy is lower (likely, and expected), explain why that's a normal, credible finding rather than a failure
6. Explain in plain language WHY the GNN outperforms XGBoost where it does – specifically because XGBoost can't see relationships between accounts (e.g. 5 accounts all sending to the same "boss" account), while the GNN can

WHY THIS MATTERS FOR THE TEAM:

This becomes the core evidence in our report that justifies "why use a GNN at all" instead of a simpler model, AND that our results generalize beyond data we made up ourselves – both are questions our supervisor/examiner will almost certainly ask. Without this, our high accuracy numbers look unverified.

I'll paste scripts/train_model.py and models/model_config.json next – please review them first, then help me write scripts that do both comparisons cleanly, and help me explain the results in report-ready language.

ROLE 2 — Backend Hardening & Testing

I'm working on a university-graded AI/ML project called TraceNet with a team of 6.

WHAT TRACENET IS:

TraceNet is an Anti-Money Laundering (AML) detection platform. It builds a graph where every bank user is a node and every transaction is an edge. A Graph Neural Network (GraphSAGE) scores how suspicious each transaction/user looks based on transaction patterns (fan-in/out, velocity, circular money flows, jurisdictions). The backend is a FastAPI server (backend/api.py, ~900 lines) with endpoints like /score_transaction, /simulate_attack, /damage_estimate, /communities, /trust. The frontend (Streamlit) calls these endpoints over HTTP.

MY SPECIFIC TASK:

Harden the backend and add real tests. This is graded as part of "software engineering practices" / code quality in our rubric, not just whether the app works.

KNOWN ISSUES IN THE CURRENT CODE (I'll paste backend/api.py so you can see exactly where):

1. There are 3 bare "except:" blocks that silently swallow ALL errors, including real bugs, not just expected failures (e.g. Ollama being offline). These need to become specific except clauses (e.g. except ConnectionError, except KeyError) with proper logging, so a real bug doesn't fail silently.
2. requirements.txt is missing packages that are actually imported in the code: "ollama" and "requests" are used but not listed. "google-generativeai" is listed but never used anywhere (dead dependency, should be removed).
3. There are currently NO automated tests anywhere in the repo.

WHAT I NEED TO BUILD:

1. Fix the bare except clauses to catch specific exceptions with proper fallback behavior (keep the graceful degradation – e.g. if Ollama is down, the report should still generate a fallback text, just not silently via a bare except)
2. Clean up requirements.txt to match actual imports
3. Write pytest test cases for the API endpoints covering:
 - Normal transaction (should be approved)
 - High-risk transaction matching a known mule pattern (should be blocked)
 - Edge cases: zero/negative amount, unknown user, malformed request body
 - Confirm risk scores are within expected 0-100 range

WHY THIS MATTERS FOR THE TEAM:

This is the part of the project that shows engineering rigor beyond "the demo works" – it's what separates a hackathon project from a graded academic one. It also protects the whole team: if backend logic breaks while others are building features on top of it, tests will catch it immediately instead of someone discovering it during the live demo.

I'll paste backend/api.py next – please review it first, then help me fix the exception handling and build out a test suite.

ROLE 3 — Frontend & New Feature

I'm working on a university-graded AI/ML project called TraceNet with a team of 6.

WHAT TRACENET IS:

TraceNet is an Anti-Money Laundering (AML) detection platform. Banks currently watch transactions in silos (mobile app, ATM, wire, UPI) – TraceNet unifies them into one transaction graph and uses a Graph Neural Network to catch money laundering patterns spanning multiple channels. When something is flagged, it's blocked and an AI writes a short report explaining why (like a compliance officer would).

The frontend is a Streamlit app (frontend/app.py, ~920 lines) that talks to a FastAPI backend over HTTP (hardcoded to http://127.0.0.1:8000 for now – I know this needs to move to a config/env variable eventually). It currently has these sections: Dashboard, Transaction Scanner, Attack Simulator, Graph Forensics (network visualization using streamlit-agraph), and SAR report viewing.

MY SPECIFIC TASK:

Polish the existing UI AND build one clearly new feature that's attributable to me specifically (this matters for individual grading, not just team output).

Feature idea – "Interbank Intelligence Sharing" view: the backend already does privacy-safe anonymization (SHA-256 hashing of identifiers, no raw PII exposed) but there's currently no dedicated frontend view showing this concept in action – e.g. a screen showing how a flagged account's data would look when shared with another bank (hashed IDs, risk score, pattern type, but no names/personal data).

WHAT I NEED TO BUILD:

1. Review the current frontend/app.py structure and identify where a new page/tab fits naturally into the existing Streamlit navigation
2. Design and build the interbank sharing view (mockup is fine if the backend endpoint doesn't fully support it yet – flag what backend support I'd need and I'll coordinate with the backend teammate)
3. General UI polish – consistent styling, loading states while waiting on API calls, clear error messages if the backend is unreachable

WHY THIS MATTERS FOR THE TEAM:

This feature directly demonstrates our project's privacy/compliance angle, which is important for an AML tool specifically – examiners will likely ask about data privacy and regulatory concerns, and having a working visual for it is much stronger than just mentioning it in the report.

I'll paste frontend/app.py next – please review its structure first, then help me plan and build this feature.

ROLE 4 — Literature Review / Related Work

I'm working on a university-graded AI/ML project called TraceNet with a team of 6. I'm not writing code for this – my job is research and the written "Related Work" section of our report.

WHAT TRACENET IS (so you understand what to research around):

TraceNet is an Anti-Money Laundering (AML) detection platform. It models bank transactions as a graph (users = nodes, transactions = edges) and uses a Graph Neural Network – specifically GraphSAGE – to detect "money mule" behavior, where criminals split laundered money across multiple accounts and channels (mobile, ATM, wire, UPI) to avoid detection by traditional siloed banking systems. It also combines the GNN score with rule-based behavioral signals (fan-in/out ratios, transaction velocity, circular money flows, high-risk jurisdictions) and uses an LLM to auto-generate Suspicious Activity Reports (SARs).

MY SPECIFIC TASK:

Find and summarize 5-10 academic papers or credible technical sources relevant to:

1. Graph Neural Networks for fraud/AML detection (especially GraphSAGE, GCN, or GAT applied to financial transaction graphs)
2. Traditional (non-graph) fraud detection methods, to explain what problem GNNs solve that older methods (e.g. XGBoost, rule-based systems) don't
3. Real-world AML systems or research (how banks currently approach this, what the \$2 trillion/year money laundering problem looks like, why cross-channel detection is a known gap)
4. Any research on using LLMs for automated compliance reporting (this is a newer, less common area – even 1-2 relevant sources helps)

WHAT I NEED TO PRODUCE:

A "Related Work" section (roughly 800-1200 words) that:

- Groups sources into these 3-4 categories, not just a list
- For each source, explains in 2-3 sentences what it did and how it differs from our approach
- Ends with a paragraph explaining the specific gap TraceNet fills (cross-channel graph-based detection + automated LLM reporting, which is a less common combination in the literature)

WHY THIS MATTERS FOR THE TEAM:

This section proves our project isn't built in a vacuum – it shows we understand the existing research landscape and can justify our technical choices (why GNN, why GraphSAGE specifically) against alternatives. This is usually a required section in any graded academic project report and is often checked closely by supervisors.

Please help me search for and summarize relevant sources, and then help me structure the writeup.

ROLE 5 — Documentation, Demo & Ethics/Limitations

I'm working on a university-graded AI/ML project called TraceNet with a team of 6.

WHAT TRACENET IS:

TraceNet is an Anti-Money Laundering (AML) detection platform. It models bank transactions as a graph and uses a Graph Neural Network (GraphSAGE) to detect money laundering patterns spanning multiple channels (mobile, ATM, wire, UPI) that traditional siloed banking systems miss. Flagged transactions are blocked and an LLM (Ollama, running locally) auto-generates a written Suspicious Activity Report explaining the decision. Reported model metrics: 99.96% accuracy, 97.87% precision, 100% recall, 98.92% F1 (on a synthetic dataset of ~5000 users, 92 labeled as mules).

MY SPECIFIC TASK (two parts):

PART A – Live demo:

There's already an undocumented script in the repo, `scripts/simulator.py`, that continuously fires random simulated transactions at the backend API and prints whether each was blocked or approved with the AI-generated reason. Nobody on the team is currently using this for anything. I need to:

1. Understand what it does (I'll paste it)
2. Turn it into a proper live demo mode – e.g. add clearer console output, maybe a way to control the rate/type of transactions, and make sure it's reliable enough to run live in front of a review panel
3. Write a short "how to run the demo" guide for the team

PART B – Report compilation + Ethics/Limitations section:

I'm assembling the final report from sections written by teammates (methodology, evaluation, related work, testing) plus writing the following sections myself:

1. Ethics & Data Privacy – TraceNet anonymizes identifiers using SHA-256 hashing before any interbank data sharing, so no raw PII is exposed. I need this explained clearly in academic language, plus a discussion of the real-world risk of false positives (a legitimate customer's transaction being wrongly blocked) and what that means for a deployed system.
2. Limitations – the dataset is synthetic (generated via the Python "Faker" library, not real bank data), which affects how much the 99.96% accuracy figure generalizes to real-world data. I need help articulating this limitation honestly without undermining the project's credibility.
3. Regulatory context – brief explanation of what a SAR (Suspicious Activity Report) is in real AML compliance, so our automated report generation feature is framed correctly against real regulatory practice.

WHY THIS MATTERS FOR THE TEAM:

The demo is often the single biggest factor in how a panel perceives the whole project, regardless of code quality – a working live demo makes everything else land better. The ethics/limitations section is also frequently required and directly graded, and being upfront about limitations (rather than ignoring them) generally reads as more credible to examiners than overclaiming.

I'll paste `scripts/simulator.py` first – please help me extend it into a clean demo tool, then help me draft the Ethics/Limitations sections.



ROLE 6 — Model / GNN Lead (your own reference, or hand off if needed)

I'm the technical lead on a university-graded AI/ML project called TraceNet, with a team of 6 members handling evaluation, backend testing, frontend, literature review, and documentation separately.

WHAT TRACENET IS:

An Anti-Money Laundering (AML) detection platform. Transactions are modeled as a graph (users = nodes, transactions = edges) across multiple channels (mobile, ATM, wire, UPI, web, branch). A GraphSAGE-based GNN (3 layers: 15 input features → 64 → 32 → 2 output classes, with batch norm and dropout) scores each node's "mule-likelihood" based on its transaction neighborhood, combined with rule-based behavioral signals (fan-in/out, velocity, circular flows, jurisdiction risk). Flagged transactions are blocked and an LLM (Ollama, qwen2.5:3b) generates a Suspicious Activity Report explaining the decision.

Current reported metrics: 99.96% accuracy, 97.87% precision, 100% recall, 98.92% F1 – on a synthetic dataset of ~5000 users with only 92 labeled mules (strong class imbalance).

MY SPECIFIC TASK:

1. Own and maintain scripts/train_model.py and the GNN architecture in backend/api.py
2. Write the "Methodology" section of the report – explaining feature engineering (the 15 input features), why GraphSAGE over alternatives like GCN or GAT, how the graph is constructed, and how the risk score combines the GNN output with rule-based signals
3. Coordinate with the teammate doing baseline comparison (XGBoost vs GNN) to make sure their evaluation script uses the exact same train/test split and features I used, so the comparison is fair and reproducible
4. Review and merge teammates' branches into main without breaking the working pipeline (data generation → training → API → frontend)

WHY THIS MATTERS FOR THE TEAM:

I'm the single point of continuity across all the moving pieces – if the model or data pipeline changes, it affects the backend, the evaluation baseline, and the demo simultaneously. Documenting my methodology clearly now also means teammates can understand and reference it in their own report sections without needing me to explain it individually each time.

I'll paste scripts/train_model.py and the GNN class in backend/api.py – help me document the methodology clearly and check for anything I should improve before teammates build on top of it.

Quick reference: who touches what file

Role	Primary file(s)
Model/GNN Lead (you)	<code>scripts/train_model.py</code> , <code>backend/api.py</code> (GNN section)
Evaluation & Baseline	<code>models/xgb_model.pkl</code> , new comparison script, Elliptic dataset download + adapter
Backend & Testing	<code>backend/api.py</code> (exception handling), <code>requirements.txt</code> , new <code>tests/</code> folder
Frontend	<code>frontend/app.py</code>
Literature Review	No code — report section only
Documentation & Demo	<code>scripts/simulator.py</code> , final report compilation

Everyone works on their own branch, pushes when done, and Aviral reviews + merges into `main`. This avoids six people editing the same files at once and keeps individual contributions traceable in git history for grading.